

Problem solving methods

Assoc. Prof. Arthur Molnar

Babeş-Bolyai University

2025

Overview

- 1 Divide and conquer
- 2 Backtracking
 - Introduction
 - Generate and test
 - Backtracking
 - Recursive and iterative
- 3 Greedy
- 4 Dynamic programming
 - Longest increasing subsequence
 - Maximum subarray sum
 - 0-1Knapsack problem
 - Egg dropping puzzle
- 5 Dynamic programming vs. Greedy

Problem solving methods

- General algorithms for solving certain types of problems
- A problem may be solved using more than one method - you have to select the most efficient one
- In order to apply one of the methods described here, the problem needs to satisfy certain criteria

Divide and conquer - steps

- **Divide** - divide the problem instance into smaller problems having the same structure
 - Divide the problem into two or more disjoint sub problems that can be resolved using the same algorithm
 - In many cases, there are more than one way of doing this
- **Conquer** - resolve the sub problems (recursively)
- **Combine** - combine the problems' results

Remember

Typical problems for Divide & Conquer

Divide and conquer - general

```
1  def divide_conquer(data):
2      if size(data) < a:
3          # solve the problem directly
4          # base case
5          return rez
6      # decompose data into d1, d2, ..., dk
7      rez_1 = divide_conquer(d1)
8      rez_1 = divide_conquer(d1)
9      ...
10     rez_k = divide_conquer(dk)
11     # combine the results
12     return combine(rez_1, rez_2, ..., rez_k)
```

Divide and conquer - general

When can divide & conquer be applied?

- Problem P on data set D may be solved by solving the same problem P on data sets $d_1, d_2, \dots, d_k$, which partition D .

The running time for solving problems in this manner may be described using recurrences.

$$T(n) = \begin{cases} \text{solve trivial problem, } n \text{ small enough} \\ k * T(\frac{n}{k}) + \text{divide time} + \text{combine time, otherwise} \end{cases}$$

Step 1 - Divide

- Simplest way: divide the data into 2 parts (*chip and conquer*): data of size 1 and data of size $n - 1$
- Example: Find the maximum

```

1  def find_max(data : list) -> int:
2      '''
3      Find the greatest element in the list
4      input: data - list of elements
5      output: maximum value
6      '''
7      if len(data) == 1:
8          return data[0]
9      # Divide into subproblems of sizes 1 and (n-1)
10     max_val = find_max(data[1:])
11     # Combine
12     return max(max_val, data[0])

```

Step 1 - Divide

- Calculating time complexity
- The recurrence is: $T(n) = \begin{cases} 1, n = 1 \\ T(n-1) + 1, \text{otherwise} \end{cases}$

$$T(n) = T(n-1) + 1,$$

$$T(n-1) = T(n-2) + 1,$$

$$T(n-2) = T(n-3) + 1, \text{ so } T(n) = 1 + 1 + 1 + \dots + 1 = n, T(n) \in \theta(n)$$

Step 1 - Divide

- Divide into k subproblems of size $\frac{n}{k}$

```

1  def find_max(data):
2      '''
3      Find the greatest element in the list
4      input: data - list of elements
5      output: maximum value
6      '''
7      if len(data) == 1:
8          return data[0]
9      # Divide into two subproblems of size n/2
10     mid = len(data) // 2
11     max_left = find_max(data[:mid])
12     max_right = find_max(data[mid:])
13     # Combine
14     return max(max_left, max_right)

```

Step 1 - Divide

- The recurrence is: $T(n) = \begin{cases} 1, n = 1 \\ 2 * T(\frac{n}{2}) + 1, \text{otherwise} \end{cases}$

$$T(2^k) = 2 * T(2^{k-1}) + 1$$

$$2 * T(2^{k-1}) = 2^2 * T(2^{k-2}) + 2$$

$$2^2 * T(2^{k-2}) = 2^3 * T(2^{k-3}) + 2^2$$

As we can divide the data into halves a number of **k** times, $n = 2^k$, then $k = \log_2 n$ and

$$T(n) = 1 + 2^1 + 2^2 + 2^3 + \dots + 2^k = \frac{2^{k+1} - 1}{2 - 1} = 2^{k+1} - 1 = 2n \in \Theta(n)$$

Divide and conquer - Example

- Compute x^k , where $k \geq 1$ is an integer
- Simple approach: $x^k = x * x * \dots * x$, $k-1$ multiplications. Time complexity?
- Divide and conquer approach

$$x^k = \begin{cases} x^{\frac{k}{2}} * x^{\frac{k}{2}}, & k \text{ is even} \\ x^{\frac{k}{2}} * x^{\frac{k}{2}} * x, & k \text{ is odd} \end{cases}$$

Divide and conquer - Example

```
1  def power(x : int, k : int) -> int:
2      '''
3      Calculate x to the power of k
4      '''
5      if k == 0:
6          return 1
7      if k == 1:
8          return x
9      # Divide
10     aux = power(x, k // 2)
11     # Conquer
12     if k % 2 == 0:
13         return aux ** 2
14     else:
15         return aux * aux * x
```

Divide and conquer - example

Problem statement

Return the product of the positive numbers found on even positions in a list, knowing the list has at least one such number.

Implementation variants:

- Product of first number times rest of the list (recursive) - chip & conquer
- Divide list into left half & right half (recursive, iterative)

Divide and conquer - applications

- Binary-Search – $\theta(\log_2 n)$
 - Divide - compute the middle of the list
 - Conquer - search on the left or for the right
 - Combine - nothing
- Exponential Search – $\theta(\log_2 n)$
 - Divide - determine the interval $[2^i, 2^{i+1}]$ where the searched value resides
 - Conquer - binary search the interval
 - Combine - nothing
- Quick-Sort – $\theta(n * \log_2 n)$
 - Divide - partition the array into 2 subarrays
 - Conquer - sort the subarrays
 - Combine - nothing

Divide and conquer - applications

- Merge-Sort – $\theta(n * \log_2 n)$
 - Divide - divide the list into 2
 - Conquer - sort recursively the 2 list
 - Combine - merge the sorted lists
- Tim Sort – $\theta(n * \log_2 n)$
 - Divide - divide the list into runs
 - Conquer - sort runs using optimized insertion sort
 - Combine - merge the sorted lists intelligently

Backtracking

- Applicable to search problems
- Generates all the solutions, if they exist
- Does a depth-first search through all possibilities that can lead to a solution
- A general algorithm/technique - must be customized for each individual application.

Remember

Backtracking usually has exponential complexity

Generate and test

- **Problem.** Let n be a natural number. Print all permutations of numbers $1, 2, \dots, n$.
- First solution - **generate & test** - generate all possible solutions and verify if they represent a solution

NB!

This is **not** backtracking

Generate and test - iterative

```
1 def perm3():
2     for i in range(0,3):
3         for j in range(0,3):
4             for k in range(0,3):
5                 # A possible solution
6                 possible_sol = [i,j,k]
7                 if i!=j and j!=k and i !=k:
8                     # Is solution
9                     print possible_sol
```

NB!

This is **not** backtracking

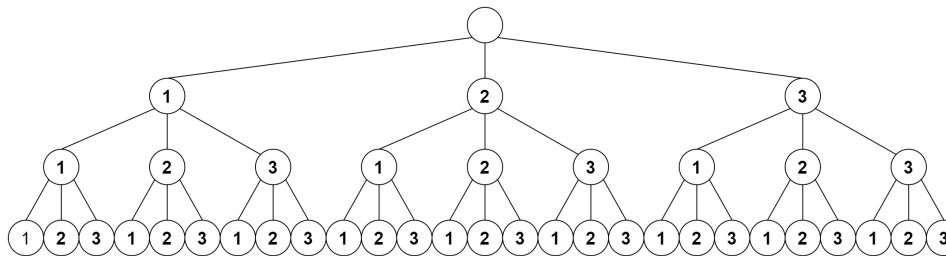
Generate and test - recursive

- Generate and test recursive - using recursion to generate all possible lists (candidate solutions)
- Still **not** backtracking but hey, at least we pretend we got rid of the loops 😊

```
1  def generate(x, DIM):
2      if len(x) == DIM:
3          print(x)
4      if len(x) > DIM:
5          return
6      x.append(0)
7      for i in range(0, DIM):
8          x[-1] = i
9          generate(x[:], DIM)
10 print(generate([], 3))
```

Generate and test

- Generate and test - all possible combinations



Generate and test

What have we learned?

- The total number of checked arrays is 3^3 , and in the general case n^n
- First the algorithm assigns values to all components of the array possible, and only afterwards checks whether the array is a permutation
- Implementation above is not general. Only works for $n=3$

Generate and test

- In general: if n is the depth of the tree (the number of variables in a solution) and assuming that each variable has k possible values, the number of nodes in the tree is k^n . This means that searching the entire tree leads to an exponential time complexity - $O(k^n)$

Generate and test

Possible improvements

- Do not construct a complete array in case it cannot lead to a correct solution.
- e.g., if the first component of the array is 1, then it is useless to assign other components the value 1
- Work with a potential array (a partial solution)
- When we expand the partial solution verify some conditions (conditions to continue) - so the array does not contains duplicates

Generate and test

Test candidates - print only solutions

```
1  def generate(x, DIM):
2      if len(x) == DIM and is_set(x):
3          print(x)
4      if len(x) > DIM:
5          return
6      x.append(0)
7      for i in range(0,DIM):
8          x[-1] = i
9          generate(x[:], DIM)
10 print(generate([], 3))
```

- We still generate all possible lists (e.g. start with 0,0,...)
- We should not explore lists that contain duplicates
- Still not **backtracking**, but hey, we only printed the solutions 😊

Backtracking

- Recursive backtracking implementation for determining permutations

```
1  def backtracking(x, DIM):
2      if len(x) == DIM:
3          print(x)
4      x.append(0)
5      for i in range(0, DIM):
6          x[-1] = i
7          if is_set(x):
8              # Continue only if we
9              # can reach a solution
10             backtracking(x[:], DIM)
11  print(backtracking([], 3))
```

Demo

Backtracking

Example illustrating exponential runtimes

[lecture.examples.ex19_backtracking.py](#)^a

^aThis is a clickable link

Backtracking - Typical problem statements

- **Permutations** - Generate all permutations for a given natural number n
 - result: $x = (x_0, x_1, \dots, x_n), x_i \in (0, 1, \dots, n - 1)$
 - is valid: $x_i \neq x_j$, for any $i \neq j$
- **n-Queen problem** - place n queens on a chess-like board such that no two queens are under reciprocal threat.
 - result: position of the queens on the chess board
 - is valid: no queens attack each other (not on the same rank, file or diagonal)
- **Latin Squares, Sudoku** - Fill a given square with symbols so they don't repeat on the row or column
 - result: the symbol in each square
 - is valid: no duplicates in the list constructed so far

Backtracking - Theoretical support

Backtracking

The key to many problems that can be solved with backtracking lays in correctly and efficiently representing the elements of the search space

Backtracking - Theoretical support

- Solutions search space: $S = S_1 \times S_2 \times \dots \times S_n$
- X is the array which represents the solutions
- $X_{[1..k]}$ in $S_1 \times S_2 \times \dots \times S_k$ is the sub-array of solution candidates; it may or may not lead to a solution
- **consistent** - function to verify if a candidate can be extended (added elements to) in order to lead to a solution
- **solution** - function to check whether the current array $x_{[1..k]}$ represents a solution

Backtracking - recursive

```
1  def backtracking(array):
2      # Add component to candidate
3      array.append(0)
4      for i in range(0, dim):
5          # Set current component
6          array[-1] = i
7          if consistent(array):
8              if solution(array):
9                  solution_found(array)
10             # Deal with next components
11             backtracking(array[:])
```

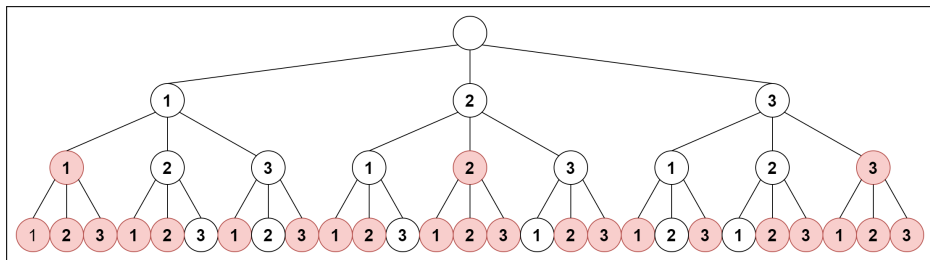
Backtracking - recursive

More generally, when solution components do not have the same domain:

```
1  def backtracking(array):
2      # Add component to candidate
3      el = first(array)
4      array.append(el)
5      while el != None:
6          # Set current component
7          array[-1] = el
8          if consistent(array):
9              if solution(array):
10                 solutionFound(array)
11                 # Deal with next components
12                 backtracking(array[:])
13             el = next(array)
```

Backtracking

The nodes explored with backtracking for the permutations of 3



Backtracking

How to use backtracking

- ① Represent the solution as a vector $X = (x_0, x_1, \dots, x_n) \in S_0 \times S_1 \times \dots \times S_n$
- ② Define what a valid solution candidate looks like (place conditions that filter out vector configurations that cannot lead to a solution)
- ③ Define the condition(s) for a candidate to be a solution
- ④ Run the backtracking template using the conditions above

Greedy

- A strategy to solve optimization problems.
- Applicable where the global optima may be found by successive selections of local optima.
- Allows solving problems without returning to previous decisions.
- Useful in solving many practical problems that require the selection of a set of elements that satisfies certain conditions (properties) and realizes an optimum.
- Disadvantages: Short-sighted and non-recoverable.

Greedy - Sample Problems

The (fractional) knapsack problem

- A set of objects is given, characterized by usefulness and weight, and a knapsack able to support a total weight of W . We are required to place in the knapsack some of the objects, such that the total weight of the objects is not larger than the given value W , and the objects should be as useful as possible (the sum of the utility values is maximal).

The coins problem

- Let us consider that we have a sum M of money and coins (ex: 1, 5, 25) units (an unlimited number of coins). The problem is to establish a modality to pay the sum M using a minimum number of coins

Greedy - General case

- Let us consider the given set C of candidates to the solution of a given problem P . We are required to provide a subset $B, (B \subseteq C)$ to fulfill certain conditions (called internal conditions) and to maximize (minimize) a certain objective function.

Greedy - General case

- If a subset X fulfills the internal conditions we will say that the subset X is acceptable (possible).
- Some problems may have more acceptable solutions, and in such a case we are required to provide as good a solution as we may get, possibly even the best one, i.e. the solution that realizes the maximum (minimum) of a certain objective function.

Greedy - General case

- The Greedy algorithm finds the solution in an incremental way, by building acceptable solutions, extended continuously. At each step, the solution is extended with the best candidate from $C - B$ at that given moment. For this reason, this method is named greedy.
- The Greedy principle (strategy) is
 - Successively incorporate elements that realize the local optimum
 - No second thoughts are allowed on already made decisions with respect to past choices.

Greedy - General case

Assuming that θ (the empty set) is an acceptable solution, we will construct set B by initializing B with the empty set and successively adding elements from C .

- The choice of an element from C , with the purpose of enriching the acceptable solution B , is realized with the purpose of achieving an optimum for that particular moment, and this, by itself, does not generally guarantee the global optimum.

Greedy - General case

- If we have discovered a selection rule to help us reach the global optimum, then we may safely use the Greedy method.
- There are situations in which the completeness requirements (obtaining the optimal solution) are abandoned in order to determine an "almost" optimal solution, but in a shorter time.

Greedy - General case

Greedy technique

- Renounces the backtracking mechanism.
- Offers a single solution (unlike backtracking, that provides all the possible solutions of a problem).
- Provides polynomial running time.

Greedy - General code

```
1 def greedy(c):
2     # The empty set is the candidate solution
3     b = []
4     while not solution(b) and c != []:
5         # Select best candidate (local optimum)
6         candidate = select_most_promising(c)
7         c.remove(candidate)
8         # If the candidate is acceptable, add it
9         if acceptable(b + [candidate]):
10             b.append(candidate)
11         if solution(b):
12             solution_found(b)
13         # In case no solution
14         return None
```

Greedy - General code (explanation)

- **c** - list of remaining candidates
- **b** - list of selected candidates (we build the solution here)
- **select_most_promising()** - returns the most promising candidate
- **acceptable()** - decides if the candidate solution can be extended
- **solution()** - verifies if a candidate represents a solution

Greedy - Essential elements

More generally, the required elements are:

- ① A **candidate set**, from which a solution is created.
- ② A **selection function**, which chooses the best candidate to be added to the solution.
- ③ A **feasibility function**, that is used to determine if a candidate can be used to contribute to a solution.
- ④ An **objective function**, which assigns a value to a solution, or a partial solution.
- ⑤ A **solution function**, which will indicate when we have discovered a complete solution.

Greedy - The coins problem

- Let us consider that we have a sum M of money and coins (ex: 1, 5, 25) units (an unlimited number of coins). The problem is to establish a modality to pay the sum M using a minimum number of coins.

Greedy - Coins problem solution

- **Candidate set** - the list of coin denominations (e.g. 1, 5, 10 bani).
- **Candidate solution** - a list of selected coins
- **Selection function** - choose the coin with the biggest value less than the remainder sum
- **Acceptable** - the total sum paid does not exceed the required sum
- **Solution function** - the paid sum is exactly the required sum

Demo

Greedy

The source code for the coins problem can be found in [lecture.examples.ex20_coins.py](#)^a

^aThis is a clickable link

Greedy - Remarks

- In the general case, we don't know whether Greedy provides an optimal solution. Often, proving this is non-trivial.
- Greedy leads to polynomial run time. Usually, if the cardinality of the set C of candidates is n , Greedy algorithms have $O(n^2)$ time complexity.

Greedy - Remarks

- There are a lot of problems that can be solved using Greedy, such as determining the shortest path between two nodes in an undirected or directed graph (Dijkstra's and Bellman-Kalaba's algorithms).

Dijkstra's Algorithm

<https://www.cs.usfca.edu/~galles/visualization/Dijkstra.html>

https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm (algorithm and visualization)

Greedy - Remarks

- There are problems for which Greedy algorithms do not provide optimal solution. In some cases, it is preferable to obtain a close to optimal solution in polynomial time, instead of the optimal solution in exponential time - these are known as *heuristic algorithms*.

Algorithms for the Traveling Salesman Problem

<https://tspvis.com/>

<https://visualgo.net/en/tsp>

Greedy example - Interval scheduling

- You want to schedule a number of jobs on a computer
- Jobs have the same value, and are characterized by their start and finish times, namely (s_i, f_i) (the start and finish times for job "i")
- Run as many jobs as possible, making sure no two jobs overlap

Greedy example - Interval scheduling

- A Greedy implementation will directly select the next job to schedule, using some criteria
- The crucial question for solving the problem - **how to determine the correct criteria?**

Source

This example, like many others can be found in "Algorithm Design", by Kleinberg & Tardos^a

^a<https://www.cs.princeton.edu/~wayne/kleinberg-tardos/>

Greedy example - Interval scheduling

Some ideas for selecting the next job

- **The job that starts earliest** - the idea being that you start using the computer as soon as possible
- **The shortest job** - the idea is to fit in as many jobs as possible
- **The job that overlaps the smallest number of jobs remaining** - we keep our options open
- **The job that finishes earliest** - we free up the computer as soon as possible

Greedy example - Interval scheduling

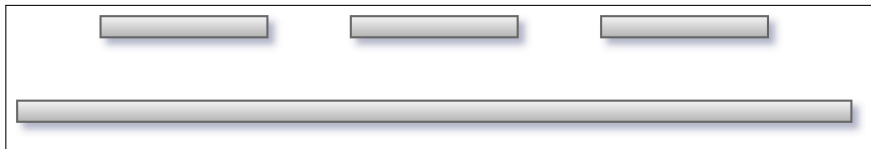
Ideas that **don't** work:

- The job that starts earliest
- The shortest job
- The job that overlaps the smallest number of jobs remaining

Greedy example - Interval scheduling

Ideas that **don't** work:

- The job that starts earliest

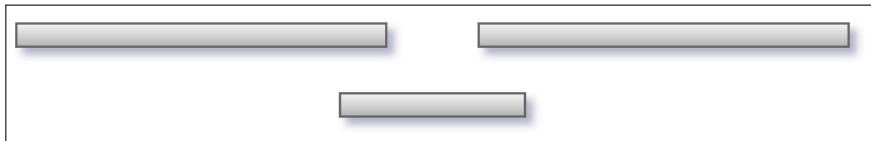


- The shortest job
- The job that overlaps the smallest number of jobs remaining

Greedy example - Interval scheduling

Ideas that **don't** work:

- The job that starts earliest
- The shortest job

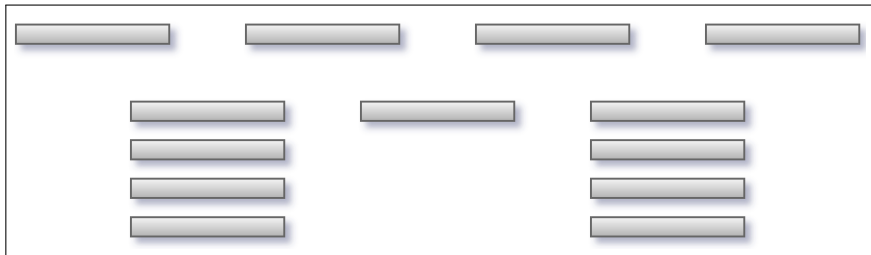


- The job that overlaps the smallest number of jobs remaining

Greedy example - Interval scheduling

Ideas that **don't** work:

- The job that starts earliest
- The shortest job
- The job that overlaps the smallest number of jobs remaining



Greedy example - Interval scheduling

An idea that **works** - Start the job that finishes earliest

S = set of jobs

while **S** is not empty:

next_job = the job that has the
soonest finishing time

add *next_job* to solution

remove from **S** jobs that overlap **q**

NB!

Proving this is done using mathematical induction, but it is beyond our scope.

Recommended learning resources

Recommended learning resources

<http://www.cs.princeton.edu/~wayne/kleinberg-tardos/>

Dynamic programming

Applicable in solving optimality problems where:

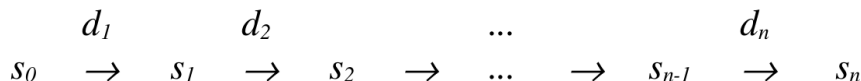
- The solution is the result of a sequence of decisions, $d_1, d_2, \dots, d_n$.
- The principle of optimality holds.

Dynamic programming:

- Usually leads to polynomial run time.
- Always provides the optimal solution (unlike Greedy).
- It combines the solutions from sub-problems (divide & conquer also does this, but differently)
- Visualize it as an optimization to applying recursion by memorizing subproblem solutions

Dynamic programming

- We consider states $s_0, s_1, \dots, s_{n-1}, s_n$, where s_0 is the initial state, and s_n is the final state, obtained by successively applying the sequence of decisions $d_1, d_2, \dots, d_n$ (using the decision d_i we pass from state s_{i-1} to state s_i , for $i=1, n$):



Dynamic programming

The method makes use of three main concepts:

- The principle of optimality
- Overlapping sub problems
- Memoization

The Principle of Optimality

Principle of Optimality

The solutions to the main problem's subproblems are themselves optimal solutions to their own subproblems 😊

- Based on the principle of optimality, the value of the optimal solution is recursively defined. This means that recurrent relations, indicating the way to obtain the general optimum from partial optima, are defined.
- The value of the optimal solution is computed in a bottom-up manner, starting from the smallest cases for which the value of the solution is known.

The Principle of Optimality

Let's take the 0-1 knapsack problem as example

- Let's say we have a knapsack of weight 20
- Let's say that putting a certain weight 10 item in will lead to an optimal solution; now we have the subproblem of filling in a weight 10 knapsack (the remaining space)
- We fill the remaining space optimally $\Rightarrow$ the solution to how to fill in the weight 10 knapsack is part of the solution to filling in the weight 20 knapsack

Now in reverse

- If you load both halves of a truck optimally, is the truck guaranteed to be loaded optimally?
- If a truck is loaded optimally, are all sections of it loaded optimally?¹

¹There's a bit of a caveat here regarding how we divide up loading space 

Overlapping Subproblems

Overlapping Subproblems

The problem can be broken down into subproblems that are overlapping, for which we calculate the optimal solution and which we assemble to get the optimal solution to the original problem.

Example

- After placing an item in the knapsack, we end up with the subproblem of a smaller knapsack and fewer items to add

Memoization

Memoization

Store the solutions to the sub-problems for later recall

A few examples

- Calculating the terms of the Fibonacci sequence, we store already calculated values
- We remember how much value we packed in all knapsack configurations in order to recall the values later

Dynamic programming - Longest increasing subsequence

Problem statement

Let us consider the list $a_1, a_2, \dots, a_n$. Determine the length as well as elements of the longest increasing subsequence $a_{i_1}, a_{i_2}, \dots, a_{i_s}$ of list a .

e.g. given sequence $[0, 8, 4, 12, 2, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15]$, a longest increasing subsequence is $[0, 2, 6, 9, 11, 15]$.

Dynamic programming - Longest increasing subsequence

- **Structure** of the optimal solution - we construct sequences:
 $L = \langle L_1, L_2, \dots, L_n \rangle$ so that for each $1 \leq i \leq n$ we have that L_i is the length of the longest increasing subsequence ending at i .
- The recursive definition for the value of the optimal solution:
 - $L_i = \max\{1 + L_j \mid A_j, A_j \leq A_i\}, \forall j = i - 1, n - 2, \dots, 1$
- **Optimality principle** - verified, solving for subarrays starting at index 0 leads to corresponding longest increasing arrays.

Demo

Longest Increasing Subsequence

[lecture.examples.ex21_longest_increasing_subsequence.py](#)^a

^aThis is a clickable link

Maximum subarray sum

Problem statement

Calculate the maximum sum of a subarray (a subarray consists of consecutive elements of the original array)

e.g. for array **[-2, -5, 6, -2, -3, 1, 5, -6]** the maximum sum of its subarrays is 7 ($6-2-3+1+5$)

Maximum subarray sum

This is a great problem, because there are several implementations.

- ① Naive implementation(s)
- ② Divide & conquer
- ③ Dynamic programming

Maximum subarray sum

Naive implementation(s)

- ① Using 3 loops: one for interval start, one for interval end, one to calculate the partial sum. At every step, compare the obtained sum with the previous maximum. This leads to $O(n^3)$ time complexity.
- ② Using 2 loops: final loop of previous implementation can be eliminated by calculating partial sums using the second loop. This leads to $O(n^2)$ time complexity.

Maximum subarray sum

Divide & Conquer implementation

- ① **Divide** - The subarray we are searching for is in one of 3 places:
 - ① Contained in left side of the array
 - ② Contained in right side of the array
 - ③ Subarray includes the middle element of the array
- ② **Conquer** - Calculate alternatives using 2 recursions and one $O(n)$ algorithm. Final complexity is $O(n * \log_2(n))$
- ③ **Combine** - Return the maximum value

Maximum subarray sum

Dynamic programming implementation

- We iterate over the array once (so $O(n)$ complexity).
- For each index, we calculate the maximum array ending at that position. If the sum is a new maximum, we record it.

Implementation

The source code for all implementations can be found at [lecture.examples.ex22_maximum_subarray_sum.py](#)^a

^aThis is a clickable link

Dynamic programming - 0-1 knapsack problem

Problem statement

You have a collection of items, each having its own weight and utility. You have a knapsack with capacity W . Which of the items can you pack in order to maximize their utility. You cannot break up items (0-1 property), and you cannot pack the same item more than once (bounded version)

0-1 knapsack problem

[lecture.examples.ex23_01knapsack.py](#)^a

^aThis is a clickable link

Dynamic programming - Egg dropping puzzle

Problem statement

Suppose you have a number of n eggs and a building having k floors. Using a minimum number of drops, determine the lowest floor from which dropping an egg breaks it.

Dynamic programming - Egg dropping puzzle

Rules:

- All eggs are equivalent.
- An egg that survives a drop is unharmed and reusable.
- A broken egg cannot be reused.
- If an egg breaks when dropped from a given floor, it will also break when dropped from a higher floor.
- If an egg does not break when released from a given floor, it can be safely dropped from a lower floor.
- You cannot assume that dropping eggs from the first floor is safe, nor that dropping from the last floor is not.

Dynamic programming - Egg dropping puzzle

The problem is about finding the correct strategy to improve the worst case outcome - make sure that the **maximum** number of drops is kept to a minimum (a.k.a minimization of maximum regret).

Dynamic programming - Egg dropping puzzle

Let's start with the simplest case...

- Building has **k** floors, and we have **n=1** egg.

Dynamic programming - Egg dropping puzzle

What do we do in the **$n=1$** case?

- Drop the egg at each floor until it breaks or you've reached the top, starting from first (ground) floor.
- In this case, the maximum number of drops is equal to k , the number of floors the building has.

Dynamic programming - Egg dropping puzzle

So, how about if we have more eggs?

- Building has k floors, but now we have $n=2$ eggs.

Discussion

How do we keep the maximum number of drops to a minimum?

Dynamic programming - Egg dropping puzzle

Possible strategies for the $n=2$ case...

- ① The $n=1$ case was basically linear search, so we can try binary search with the first egg (drop it from the mid-level floor).
- ② How about dropping from every 20th floor, starting from ground level?

Dynamic programming - Egg dropping puzzle

Let's try to describe the situation in a structured way...

- Imagine we drop the first egg at a given floor **m**.
- If it breaks, we have a maximum of **(m-1)** drops, starting from ground.
- If it does not break, we increase by **(m-1)** floors, as we have one less drop.
- Following the same logic, at each step we decrease the number of floors by 1.

Dynamic programming - Egg dropping puzzle

So, if we have **n=2** eggs and **k=100** floors?

- We solve $m + (m - 1) + (m - 2) + (m - 3) + (m - 4) + \dots + 1 \geq 100$
- Solution is between 13 and 14, which we round to **14**. First drop is from floor 14

Dynamic programming - Egg dropping puzzle

Egg drops for **n=2** eggs and **k=100** floors...

Drop #	1	2	3	4	5	6	7	8	9	10	11	12
Floor	14	27	39	50	60	69	77	84	90	95	99	100

Dynamic programming - Egg dropping puzzle

Now for the general case - building has k floors, and we have n eggs.

- **Optimal substructure** - Dropping an egg from floor x might result in two cases (subproblems):
 - ① **Egg breaks** - Problem is reduced to one with $x-1$ floors and one fewer egg.
 - ② **Egg is ok** - Problem is reduced to one with $k-x$ floors and same number of eggs.

Dynamic programming - Egg dropping puzzle

- **Overlapping subproblems** - Let's create function **eggDrop(n, k)**, where **n** is the number of eggs and **k** the number of floors.
- **eggDrop(n, k) = 1 + min{max(eggDrop(n - 1, x - 1), eggDrop(n, k - x))}**, with $1 \leq x \leq k$

Demo

Egg dropping puzzle

[lecture.examples.ex24_egg_dropping_puzzle.py](#)^a

^aThis is a clickable link

Dynamic programming vs. Greedy - Remarks

- Both techniques are applied in optimization problems
- Greedy is applicable to problems for which the general optimum is obtained from partial (local) optima
- Dynamic programming is applicable to problems in which the general optimum implies partial optima.

Dynamic programming vs. Greedy - An example

Problem statement

Let us consider the problem to determine the optimal path between two vertices i and j of a graph.

What we notice:

- The principle of optimality is verified: if the path from i to j is optimal and it passes through node x , then the paths from i to x , and from x to j are also optimal.
- Can we apply Greedy? - if the paths from i to x , and from x to j are optimal, there is no guarantee that the path from i to j that passes through x is also optimal

Dynamic programming vs. Greedy - Remarks

- The fact that the general optimum implies partial optima, does not mean that partial optima also implies the general optimum (the example before, or the one with loading the two halves of a truck are relevant).
- The fact that the general optimum implies partial optima is very useful, because we will search for the general optimum among the partial optima, which are stored at each moment. Anyway, the search is considerably reduced.